UNE QUESTION AVANT DE COMMENCER

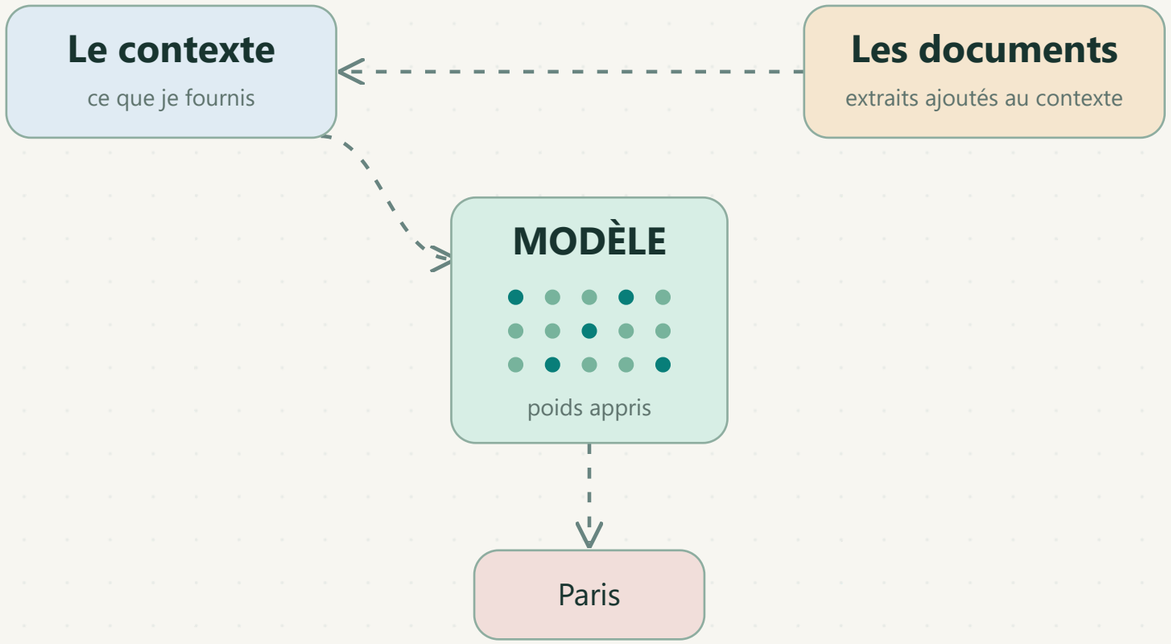
Où se cache ce que sait une IA ?

Un grand modèle de langage (LLM), une forme d'intelligence artificielle, apprend à prévoir la suite d'un texte.

- **Poids :** nombres appris qui règlent les calculs.
- **Contexte :** texte reçu pour cette réponse.
- **Documents :** textes externes qu'un outil peut fournir.

À RETENIR

Ces trois sources ne jouent pas le même rôle. Suivons leur utilisation.



Les extraits consultés rejoignent le contexte.

Exemple : « La capitale de la France est… ». Consulter un document ne modifie pas automatiquement les poids.

DEUX MOMENTS À NE PAS CONFONDRE

Apprendre. Puis utiliser.

L'apprentissage ajuste les poids à partir d'exemples. L'inférence utilise ces poids pour produire une sortie.

> Pendant une inférence ordinaire, les poids restent fixes.

> Une application peut garder la conversation sans réentraîner le modèle.

À RETENIR

Ajouter du contexte et apprendre de nouveaux poids sont deux opérations différentes.

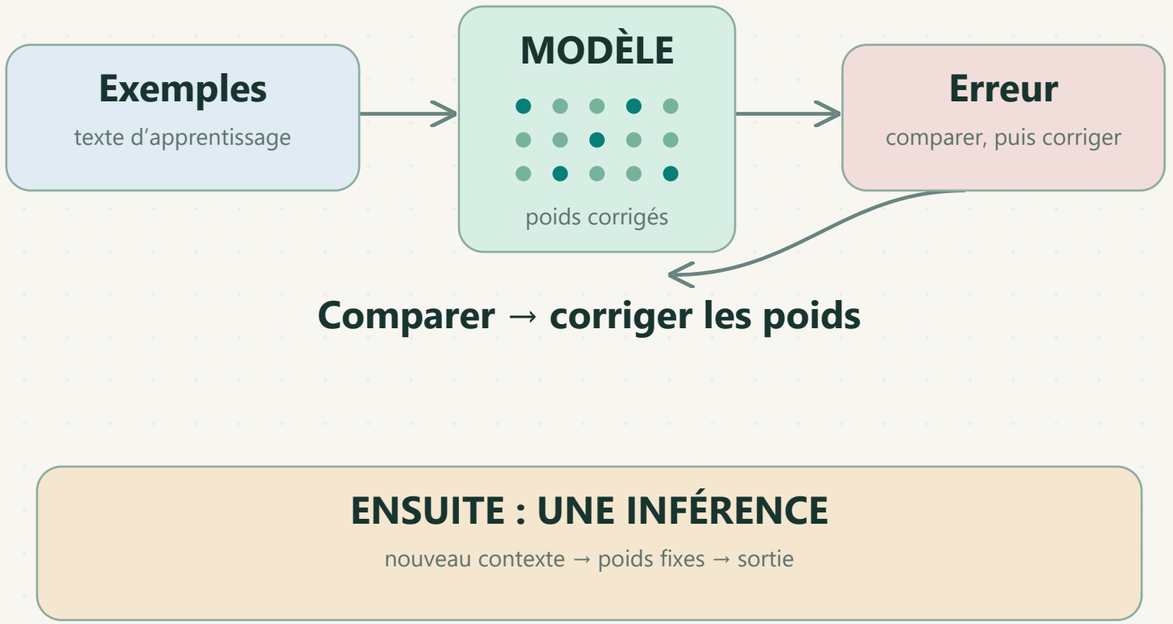


Schéma simplifié. Une application peut garder un historique sans modifier les poids.

DU TEXTE AUX NOMBRES

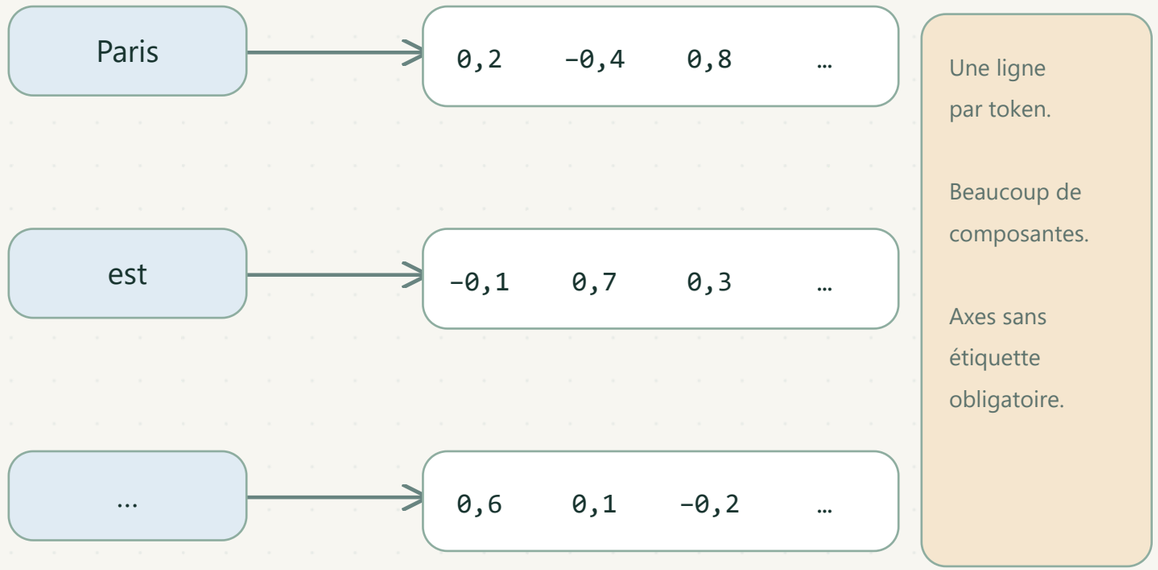
Un morceau de texte devient un vecteur.

Un token est un morceau de texte. Un vecteur est une liste ordonnée de nombres.

> **Tokenizer :** outil qui découpe le texte en tokens.

> **Embedding :** vecteur appris associé à un token.

> Un axe est un repère ; il ne signifie pas forcément « joie », « animal » ou « passé ».



Tokens et coordonnées dessinés pour expliquer ; ce ne sont pas les sorties d'un tokenizer ou d'un LLM.

À RETENIR

Le réseau transforme ensuite ces vecteurs en tenant compte du contexte.

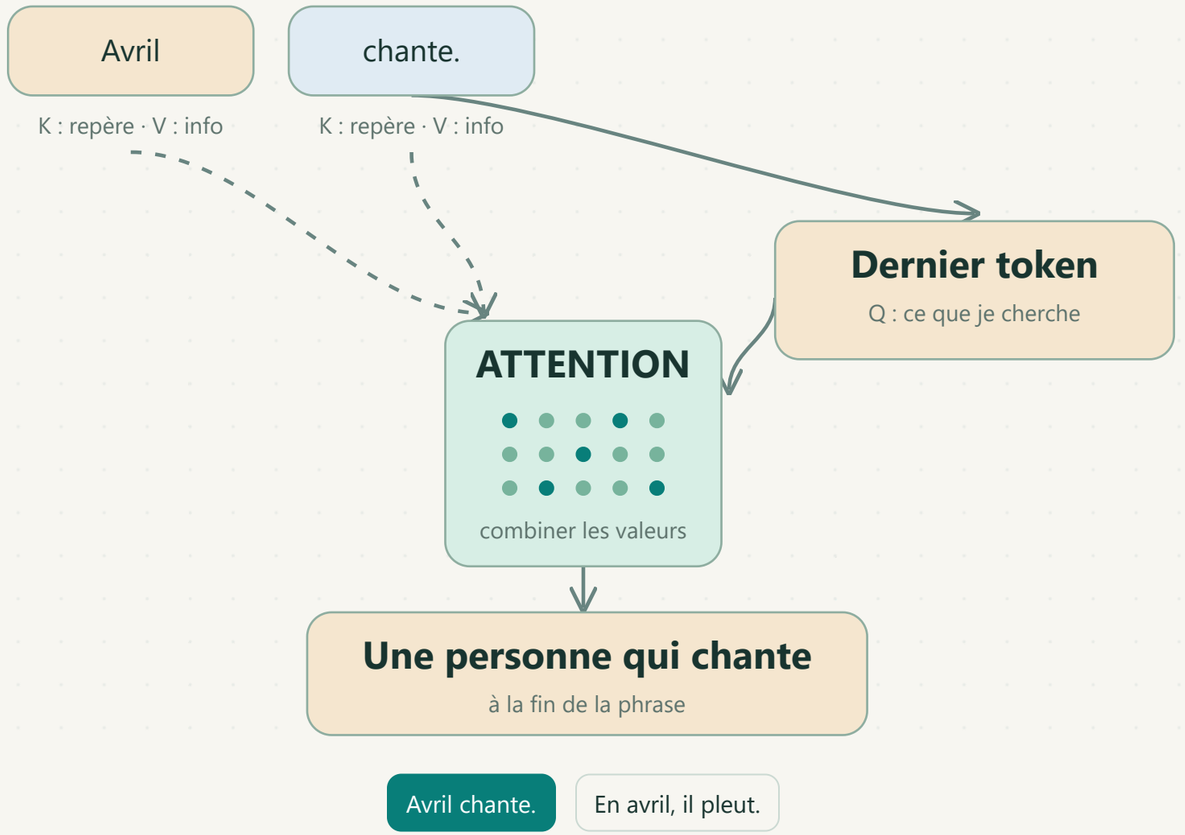
LE CONTEXTE CHANGE L'INTERPRÉTATION

« Avril » : qui ou quand ?

Un Transformer enchaîne des blocs d'attention, qui combinent le contexte, et des transformations de vecteurs.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

Q : requêtes, ce que l'on cherche ; K : clés à comparer.
V : valeurs, informations à combiner ; d_k : taille d'une clé.
^{T} échange lignes et colonnes. softmax donne des pondérations positives dont la somme vaut 1.



Interprétation illustrée à la fin de la phrase, pas au token « Avril » isolé. Liens dessinés, pas mesurés dans un modèle.

À RETENIR

Un modèle causal ne lit pas de token futur. À la fin de la phrase, toute la phrase est déjà disponible.

UNE REPRÉSENTATION PARTAGÉE

Un neurone n’est pas un tiroir à faits.

Un neurone combine des entrées. Les poids se rangent en matrices, des tableaux de nombres.

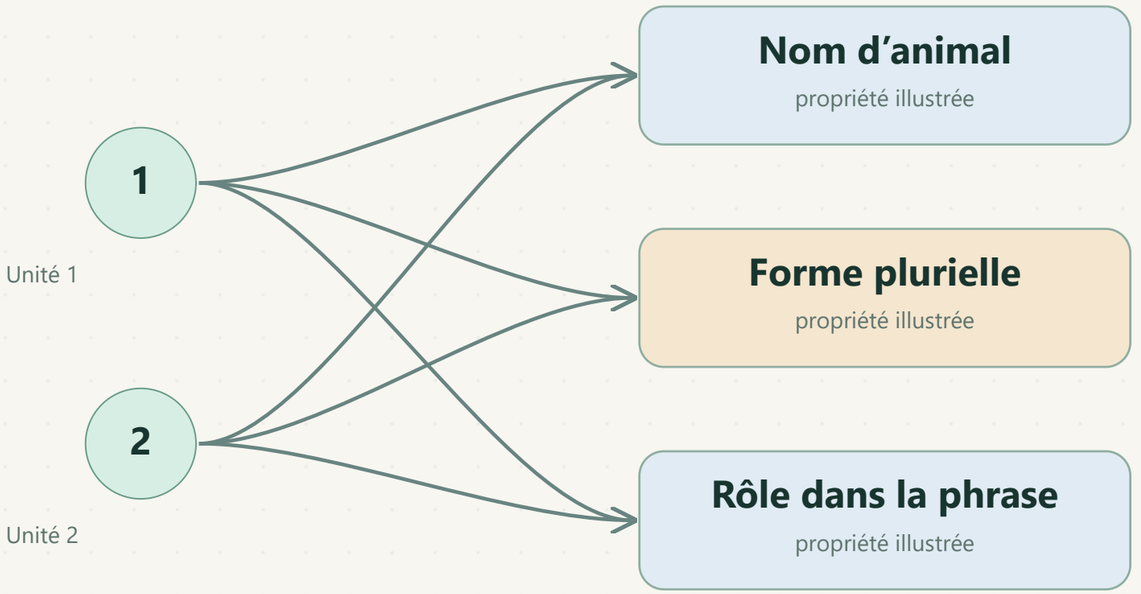
> **MLP :** perceptron multicouche, un réseau de couches de neurones.

> **Activation :** fonction qui transforme le résultat d’un neurone.

> **Superposition :** des propriétés partagent des dimensions de représentation.

À RETENIR

Une propriété peut mobiliser plusieurs unités. Ce partage n’empêche pas la mémorisation de passages.



Deux unités partagées par trois propriétés illustrées.

Deux unités et trois propriétés illustrées : ce partage peut créer des interférences. Ce n’est pas une carte réelle de neurones.

À VOUS DE PRÉDIRE

J’ajoute un nom dans mon message.

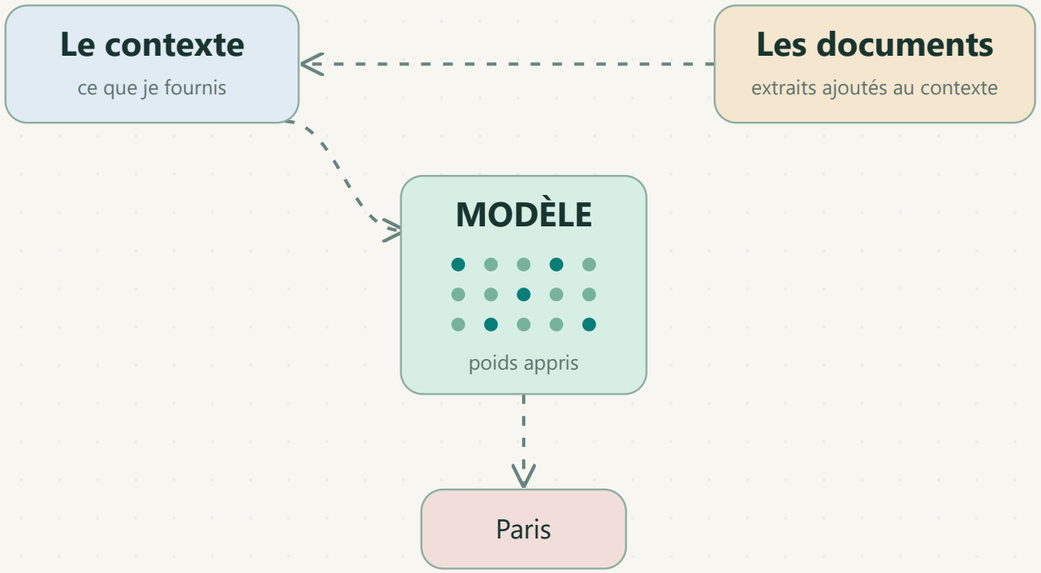
Le modèle réutilise ce nom. Quelle explication suffit sans supposer de réentraînement ?

A. Le contexte peut suffire à expliquer sa réponse.

B. Les poids ont été modifiés par ce message.

C. Le nom a été enregistré durablement pour tous.

La réponse peut utiliser le contexte. À elle seule, elle ne prouve ni mise à jour des poids ni mémoire durable.



Les extraits consultés rejoignent le contexte.

À RETENIR

Poids appris, contexte temporaire et stockage de l’application sont trois choses différentes.

Une réponse correcte ne suffit pas à identifier le mécanisme de mémorisation.

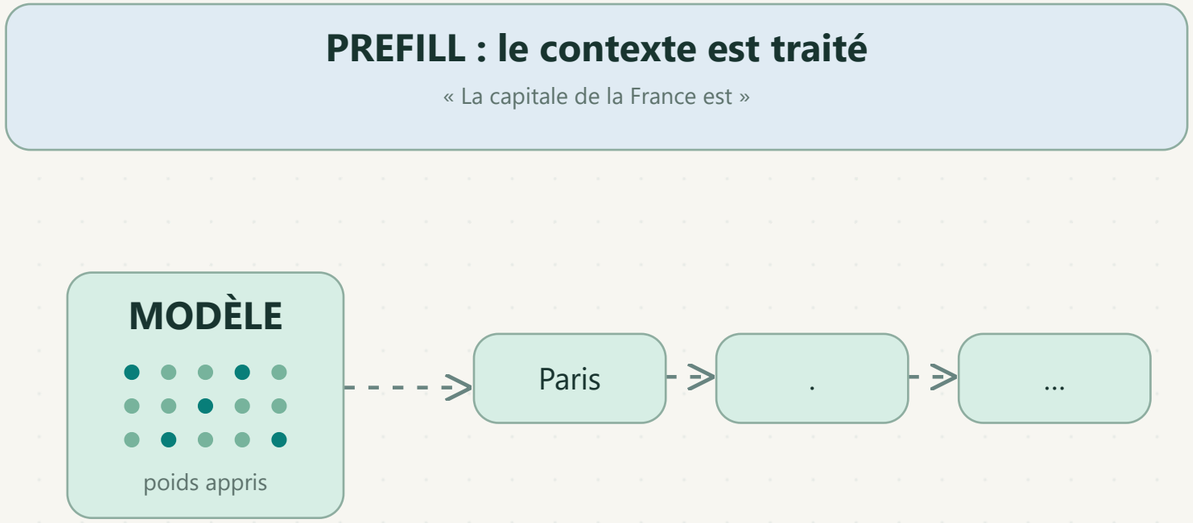
DEUX PHASES DE L'INFÉRENCE

Lire le début. Puis continuer.

Prefill : traiter le contexte. Decode : produire la suite, token après token.

> **Débit :** nombre de tokens produits par seconde.

> Génération **autorégressive** : chaque nouveau token dépend de ceux déjà présents.



DECODE : une suite qui se construit.

À RETENIR

Mesurer séparément le délai jusqu'au premier token et le débit des tokens suivants.

Génération ordinaire. Une technique vue plus loin vérifiera plusieurs propositions ensemble.

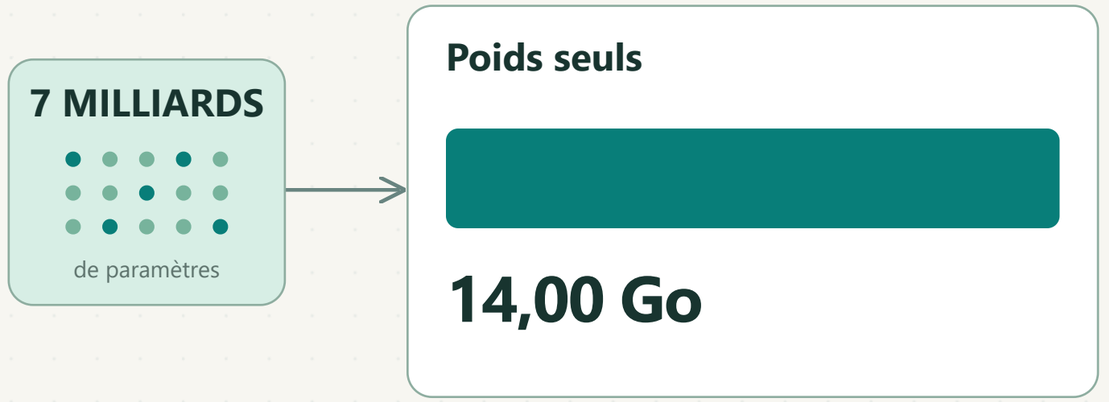
PREMIER BUDGET : LES POIDS

Sept milliards de nombres.

Un bit vaut 0 ou 1. Huit bits font un octet. La précision choisie change le nombre de bits par poids.

$$M_{poids} = N \times b / 8$$

N : nombre de paramètres, les réglages appris.
b : bits par paramètre ; M : taille en octets.
Ici $N = 7 \times 10^9$; 1 Go $= 10^9$ octets.



16 bits par paramètre · compte idéal

À RETENIR

Le programme, ses données de travail et le format du fichier occupent aussi de la mémoire.

16 bits | 8 bits | 4 bits | 2 bits

Poids seuls : ni la RAM totale (mémoire de travail), ni un fichier complet. La barre change avec les bits.

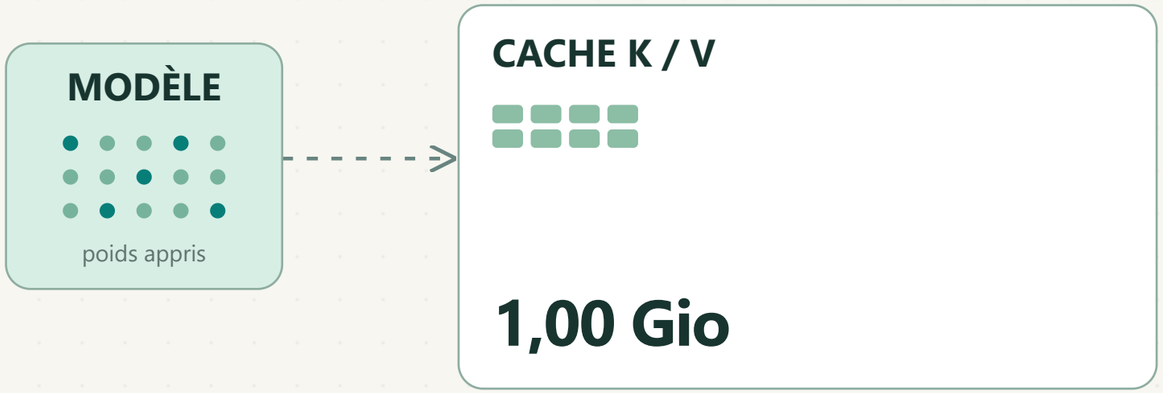
DEUXIÈME BUDGET : LE CONTEXTE

Le cache grandit avec l'histoire.

Un cache est une mémoire de réutilisation. Celui de l'attention garde les clés K et valeurs V déjà calculées.

$$M_{KV} = 2 \times L \times S \times H_{KV} \times d \times p$$

Pour une séquence : L couches, S tokens,
H_{KV} têtes K/V, d composantes par tête,
p octets par composante. Le 2 compte K et V.



8 192 tokens · 8 têtes K/V

Plus de contexte : colonnes · plus de têtes : lignes

Tokens 8 192 8 têtes K/V · changer

L=32, d=128, p=2. GQA : attention à requêtes groupées. Cases regroupées, sans surcoûts ; 1 Gio=2^{30} octets.

À RETENIR

Une tête est une branche d'attention. En GQA, plusieurs têtes de requêtes partagent les mêmes K/V.

LE CALCUL ATTEND PARFOIS LES DONNÉES

Une usine rapide. Une route étroite.

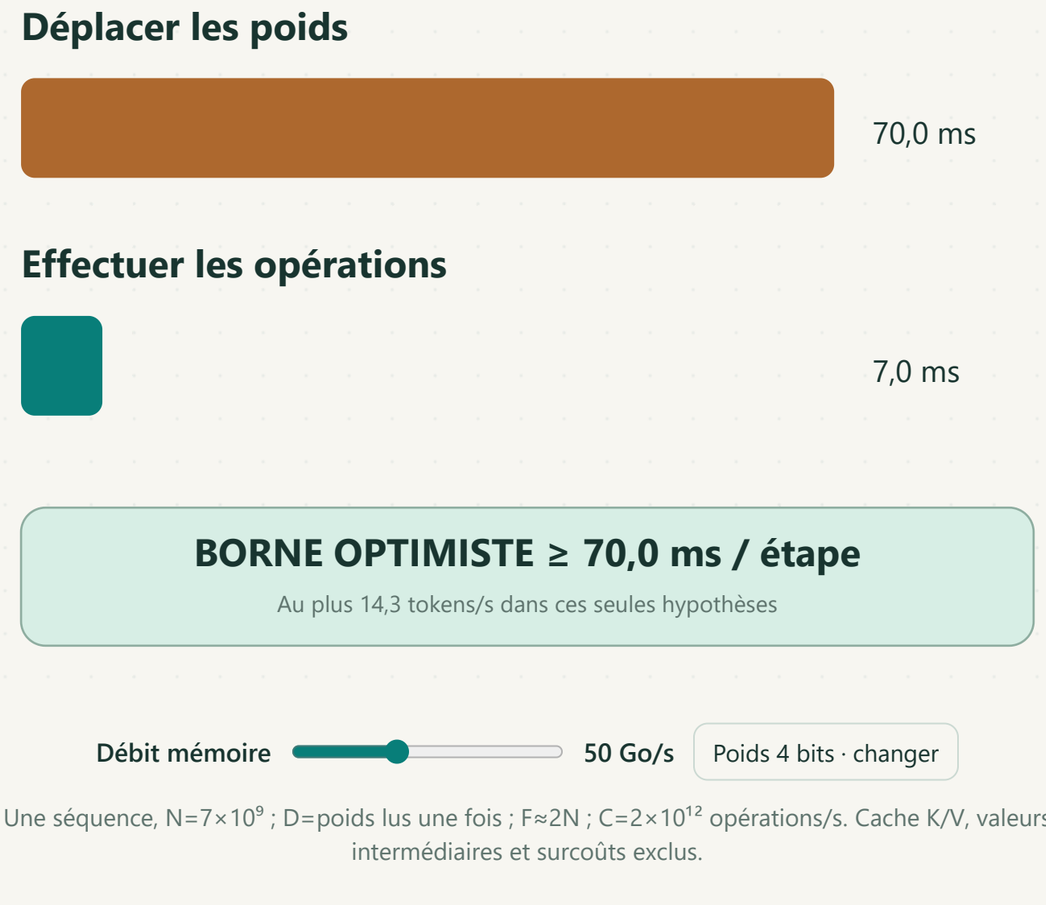
Le débit mémoire est la quantité de données transférable par seconde. Le calcul peut attendre ces données.

$$T \geq \max(D / B, F / C)$$

T : latence, durée d'une étape ; max retient le plus grand temps.
D : octets transférés ; B : débit mémoire en octets/s.
F : opérations ; C : capacité de calcul en opérations/s.

À RETENIR

Cette borne est un minimum théorique. Elle n'est pas une durée mesurée sur une machine.



Une séquence, $N=7\times10^{9}$; D=poids lus une fois ; $F\approx2N$; $C=2\times10^{12}$ opérations/s. Cache K/V, valeurs intermédiaires et surcoûts exclus.

UN DIAGNOSTIC AVANT UNE OPTIMISATION

Le calcul prend 7 ms. La lecture, 70 ms.

Dans le modèle idéal, on double seulement la capacité de calcul. Que devient la borne de temps ?

A. Elle passe à 35 ms.

B. Elle reste à 70 ms : les transferts dominent.

C. Elle passe à 7 ms.

La borne max(70,7) reste 70 ms quand 7 devient 3,5 ms. Une machine réelle ajoute d'autres coûts.



Nombres d'un exemple analytique, pas d'une mesure de PC ou de Raspberry Pi.

À RETENIR

Optimiser une brique peu coûteuse peut presque ne rien changer au temps total.

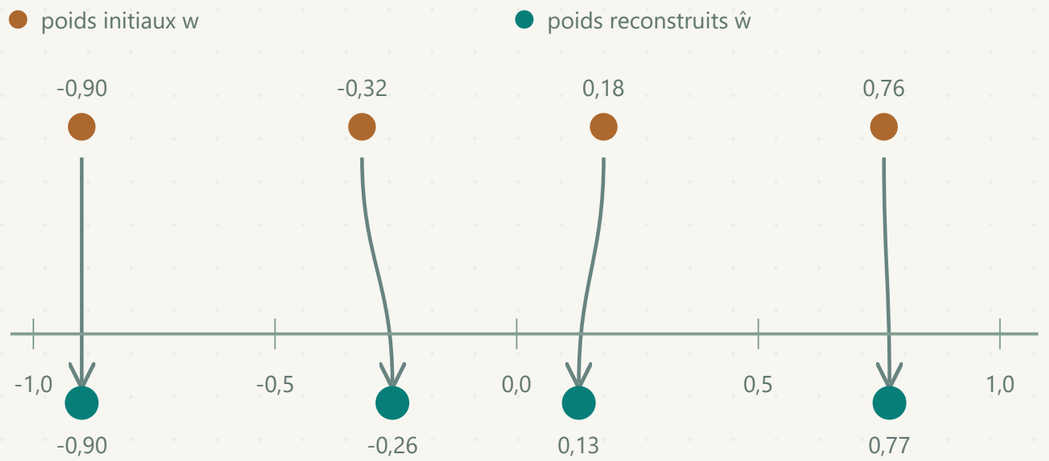
PISTE 1 · REPRÉSENTER AUTREMENT

Arrondir les poids. Observer la sortie.

Quantifier : remplacer des valeurs par quelques niveaux.

$$s = \max|w| \,/\, q_{max}$$
$$q = \text{borne}(\text{arrondi}(w/s), -q_{max}, q_{max})$$
$$\hat{w} = s \times q$$

b : bits ; $q_{max}=2^{b-1}-1$: plus grand code positif.
s : échelle, pas entre niveaux ; max|w| : plus grand poids sans signe.
w : poids ; q : code entier ; ŵ : poids reconstruit.
x : entrée ; y : sortie ; i : indice. Si max|w|=0 : s=1.



Sortie initiale : -1,0500
y = somme des $w_i x_i$

Après : -0,9643
écart absolu : 0,0857

8 bits | 4 bits | 2 bits

w=(−0,9 ; −0,32 ; 0,18 ; 0,76), x=(1 ; 0,5 ; −1 ; 0,25). Borner ramène au code autorisé ; arrondi à mi-distance loin de zéro.

À RETENIR

Calcul en flottants (réels approchés) : le stockage réduit ne promet pas d'accélération.

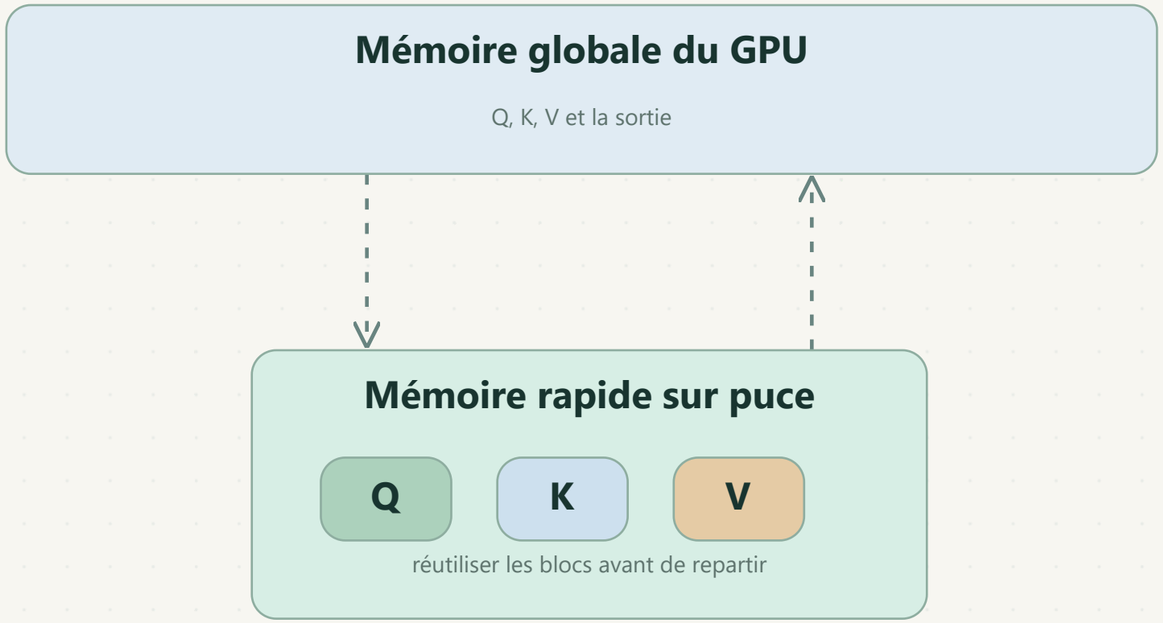
PISTE 2 · CHANGER LE TRAJET DES DONNÉES

Même attention. Moins d’allers-retours.

Un GPU est un processeur qui effectue beaucoup de calculs en parallèle. FlashAttention réorganise ses accès mémoire.

> La mémoire rapide est petite : le calcul est organisé en blocs.

> Réutiliser un bloc évite des lectures répétées en mémoire globale.



Tri Dao et ses coauteurs, 2022. Schéma de principe GPU ; aucun gain automatique n’est promis sur Raspberry Pi.

À RETENIR

L’attention reste complète, à l’arrondi numérique près. FlashAttention ne supprime pas le cache K/V.

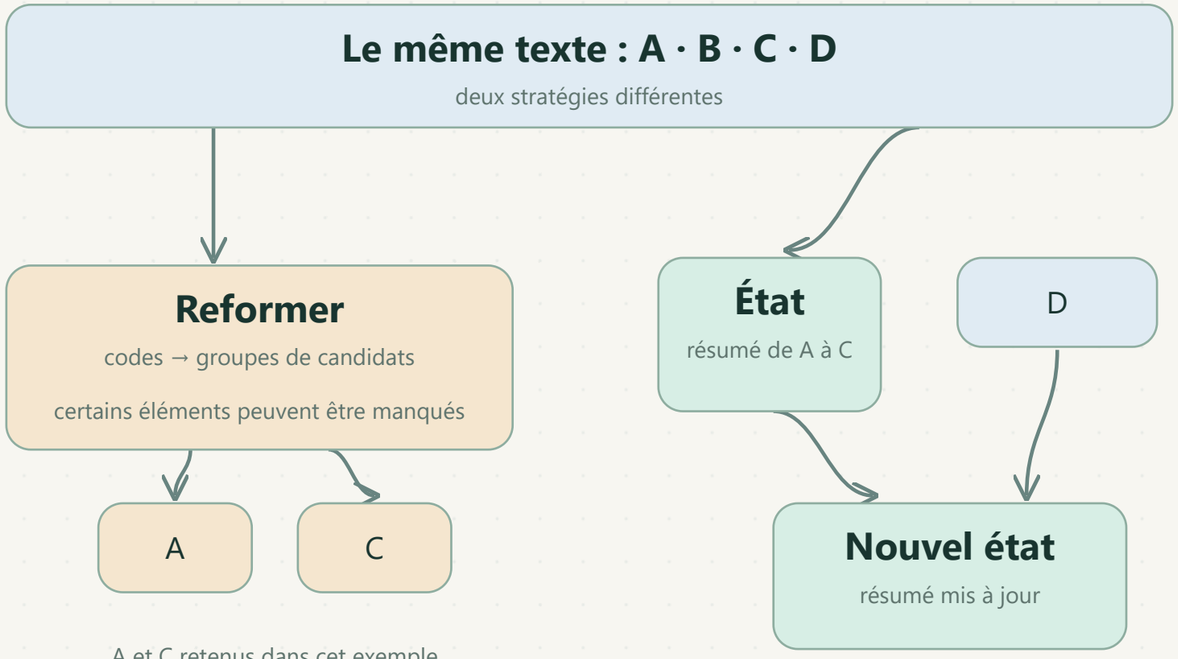
PISTE 3 · CHANGER LA MÉMOIRE DU MODÈLE

Garder des éléments. Ou résumer le passé.

Une autre architecture change la manière de conserver et retrouver le passé.

> **Reformer · 2020 :** le hachage calcule des codes pour regrouper des éléments proches. Certains éléments peuvent être manqués.

> **Mamba · 2023 :** un état est un résumé numérique ; la récurrence le met à jour à chaque token.



À RETENIR

Pour un modèle fixé, l'état de Mamba ne grandit pas avec le contexte. Cela ne préserve pas forcément tous les détails.

Kitaev et al. pour Reformer ; Albert Gu et Tri Dao pour Mamba. Ce ne sont pas des remplacements universellement meilleurs.

LA QUESTION QUI ÉVITE UNE FAUSSE CONCLUSION

Deux textes ont le même petit code.

Peut-on garantir qu'on retrouvera toujours l'information exacte avec ce code seul ?

A. Oui : un même code implique un même texte.

B. Oui : si le code est produit de façon déterministe.

C. Non : des informations différentes peuvent partager un code.

Une collision signifie que des entrées différentes ont le même code. Un calcul déterministe, qui donne toujours le même code à une même entrée, n'empêche pas ces collisions.

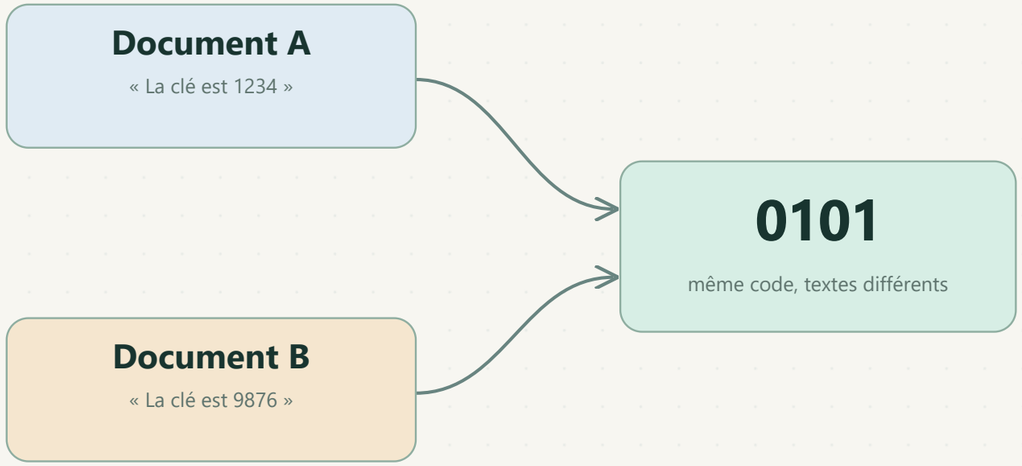


Schéma de collision construit pour expliquer. Il ne mesure pas le taux de collision d'une méthode publiée.

À RETENIR

Un coût réduit doit être accompagné d'une vérification de ce qui a été perdu.

PISTE 4 · ENTRAÎNER POUR UNE AUTRE ARITHMÉTIQUE

Et avec des poids −1, 0, +1 ?

Ternaire signifie trois valeurs possibles. BitNet b1.58 apprend avec des poids −1, 0 et +1.

$$(-1)\times x_1 + 0\times x_2 + (+1)\times x_3$$
$$= -x_1 + x_3$$

x_1, x_2, x_3 : trois valeurs d'entrée.
$\log_2(3) \approx 1{,}58$ bits : limite idéale pour trois choix aussi fréquents.
Échelles et autres données s'ajoutent au stockage réel.

−1
soustraire

0
ignorer

+1
additionner

$$y = -x_1 + x_3$$

Exemple de poids ternaires ; activations non binaires.

À RETENIR

Un noyau est une fonction de calcul adaptée au processeur. Entraînement et noyaux doivent être compatibles.

Ma et al., 2024 ; rapport 2B4T, 2025. Le calcul ternaire ne s'obtient pas en arrondissant arbitrairement un modèle existant.

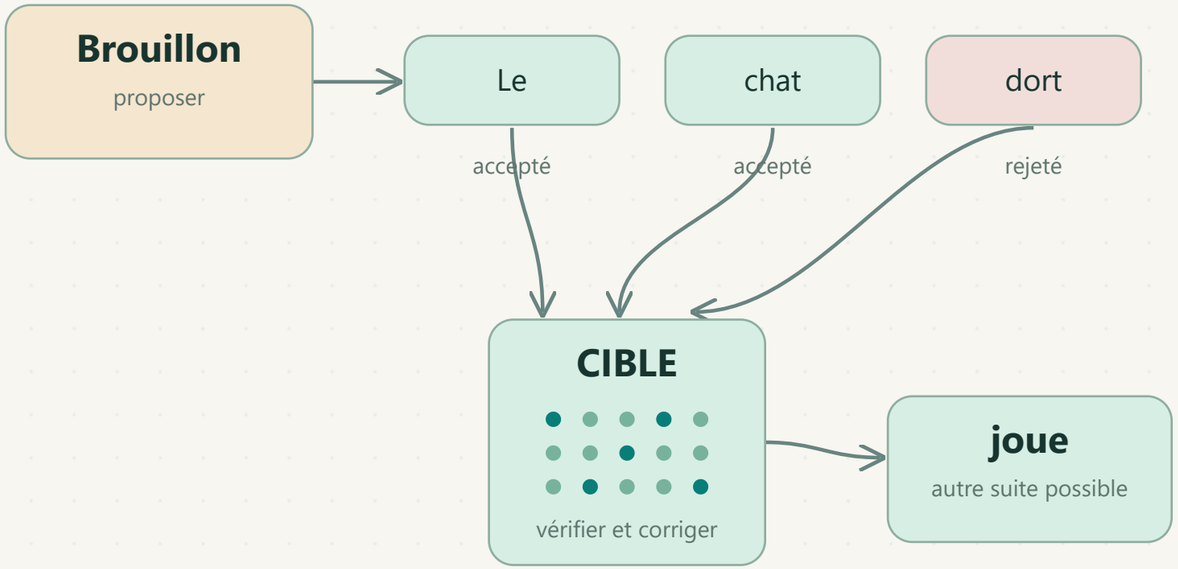
PISTE 5 · PROPOSER, PUIS VÉRIFIER

Un petit modèle écrit. Le grand contrôle.

Le modèle brouillon propose une suite. Le modèle cible, celui que l'on veut reproduire, la vérifie.

> **Distribution :** probabilités attribuées aux tokens possibles.

> **Échantillonner :** tirer un token selon ces probabilités. Une correction préserve la distribution de la cible dans l'algorithme exact.



La vraie règle porte sur des probabilités, pas sur « cela semble juste ».

À RETENIR

Vérifier plusieurs propositions ensemble peut faire gagner du temps. Le coût du brouillon peut aussi annuler ce gain.

Leviathan et al., ICML 2023. Deux suites plausibles : la vérification porte sur les probabilités, pas sur une vérité.

DISTINGUER LES NIVEAUX DE PREUVE

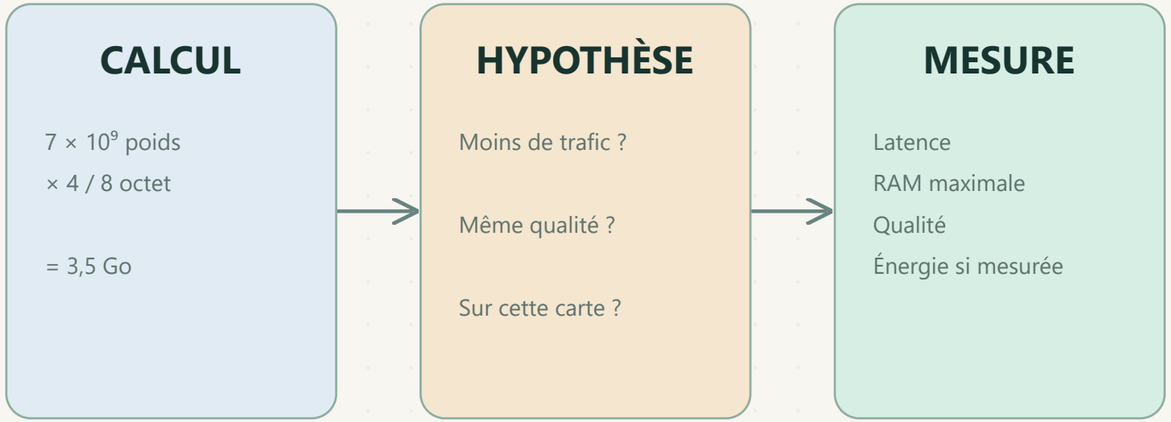
Une équation n’est pas un chronomètre.

Un calcul peut aider à choisir une piste. Il ne démontre pas la qualité d’un modèle ni sa vitesse réelle.

> **Hypothèse :** résultat attendu, écrit avant les mesures.

> **Calculé :** taille des poids ou minimum théorique de temps.

> **Mesuré :** durée, RAM maximale et qualité sur des exemples de test.



Un débit et un score de qualité exigent des expériences.

À RETENIR

Une taille de fichier ne démontre ni un débit réel ni une qualité de réponse.

Qualité : réussite sur la tâche choisie. Le jeu de test reste indépendant des données qui ont servi à choisir les réglages.

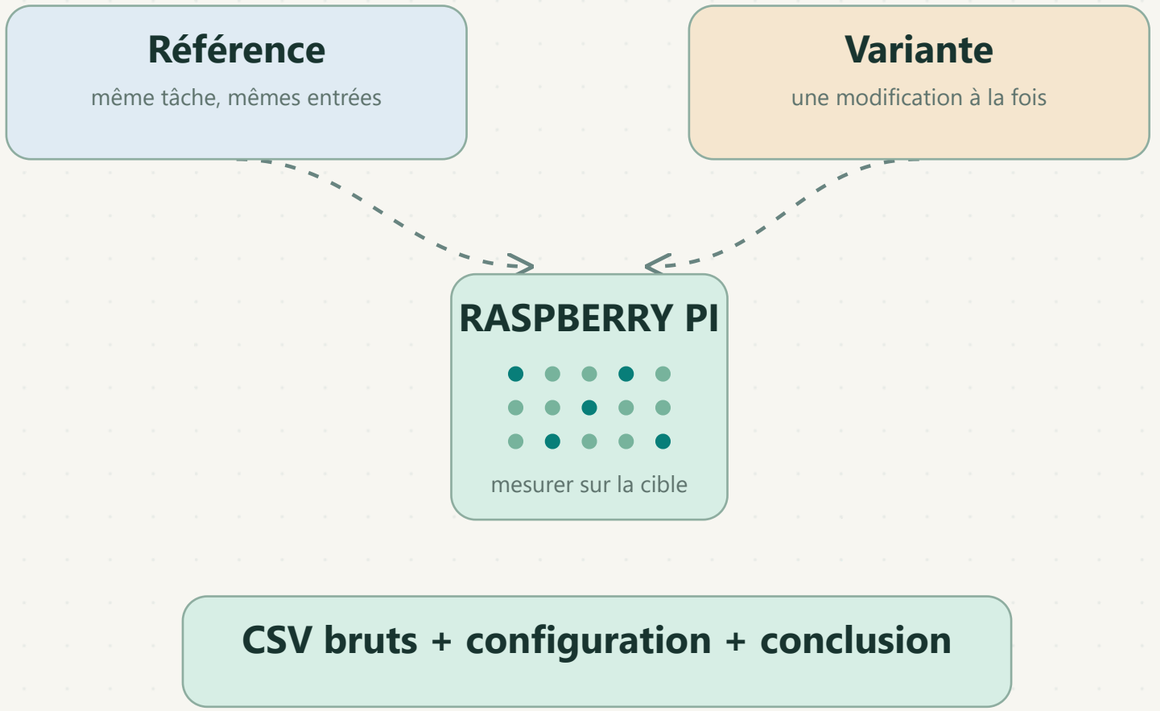
DU PC À LA MACHINE VISÉE

Sur le Pi : même tâche, vraies mesures.

Le runtime est le logiciel qui exécute le modèle. Fixer aussi le Pi, les entrées et les réglages.

> **Référence :** version de départ ; **variante :** une seule brique modifiée.

> Faire des essais de chauffe, puis répéter et alterner les mesures. Séparer chargement, premier token et suite.



Chauffe : appels avant la mesure. Empreinte : identifiant calculé sur un fichier. Mesurer sur le Pi ; sans instrument, pas de conclusion sur l'énergie.

À RETENIR

Garder les mesures brutes en CSV, un fichier tableau, et l' empreinte des fichiers pour identifier exactement ce qui a été testé.

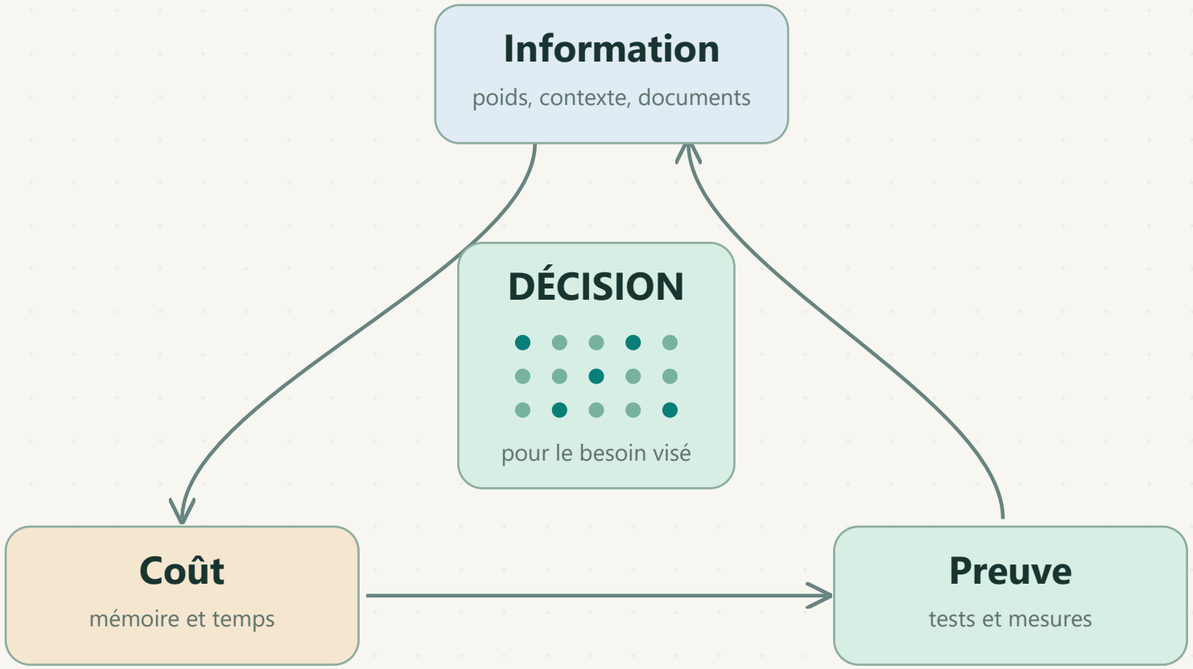
TROIS RÉFLEXES D’INGÉNIEUR

Où est l’information ? Quel est le coût ? Quelle est la preuve ?

Une optimisation utile respecte le besoin de l’utilisateur et les limites de la machine.

> Les poids portent des régularités apprises ; le contexte et les documents apportent des informations.

> Moins de bits, moins de transferts ou une autre architecture : les compromis doivent être vérifiés.



À RETENIR

Choisir une seule brique, écrire une hypothèse et comparer à la référence sur la tâche visée.

Les trois pages suivantes donnent les sources et leurs limites. Le cours complet et le TD restent accessibles dans les ressources.

TEXTES ORIGINAUX

Représenter, apprendre, interpréter.

Des sources pour préciser ce que les schémas simplifient.

R1 · Attention Is All You Need ↗

Ashish Vaswani et al. · 2017
NeurIPS 2017

R2 · Locating and Editing Factual Associations in GPT ↗

Kevin Meng, David Bau, Alex Andonian, Yonatan Belinkov · 2022
NeurIPS 2022

R3 · Toy Models of Superposition ↗

Nelson Elhage et al. · 2022
Article de recherche · modèles jouets

R4 · Extracting Training Data from Large Language Models ↗

Nicholas Carlini et al. · 2021
USENIX Security 2021 · prépublication initiale 2020

R16 · Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ↗

Patrick Lewis et al. · 2020
NeurIPS 2020

À RETENIR

Les mécanismes observés dans une expérience ne constituent pas une carte complète de tous les modèles.

TEXTES ORIGINAUX

Compter les octets.
Changer les calculs.

Un gain publié garde le contexte matériel et le protocole du papier.

R5 · Roofline: An Insightful Visual Performance Model… ↗

Samuel Williams, Andrew Waterman, David Patterson · 2008 / 2009
Rapport Berkeley 2008 · article CACM 2009

R6 · GQA: Training Generalized Multi-Query Transformer Models… ↗

Joshua Ainslie et al. · 2023
EMNLP 2023

R7 · AWQ: Activation-aware Weight Quantization… ↗

Ji Lin et al. · 2023 / 2024
Prépublication 2023 · MLSys 2024

R8 · FlashAttention: Fast and Memory-Efficient Exact Attention… ↗

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré · 2022
NeurIPS 2022

R9 · Reformer: The Efficient Transformer ↗

Nikita Kitaev, Łukasz Kaiser, Anselm Levskaya · 2020
ICLR 2020

R10 · Mamba: Linear-Time Sequence Modeling with Selective State Spaces ↗

Albert Gu, Tri Dao · 2023 / 2024
Prépublication 2023 · COLM 2024

À RETENIR

Les simulations de cette présentation sont des exemples pédagogiques ; aucun LLM n’a été exécuté sur Pi ici.

ARTICLES ET IMPLÉMENTATIONS

Des pistes réelles. Des limites à vérifier.

Sélection de publications et de code ; ce n’est pas un classement exhaustif de la recherche.

R11 · The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits ↗

Shuming Ma et al. · 2024
Prépublication · résultats dans les configurations étudiées

R12 · BitNet b1.58 2B4T Technical Report ↗

Shuming Ma et al. · 2025
Rapport technique · prépublication

R13 · Fast Inference from Transformers via Speculative Decoding ↗

Yaniv Leviathan, Matan Kalman, Yossi Matias · 2023
ICML 2023 · prépublication initiale 2022

R14 · GGUF — spécification du format ↗

Projet ggml · Documentation évolutive
Documentation officielle

R15 · BitNet — implémentation officielle ↗

Microsoft Research · Code évolutif
Dépôt officiel

À RETENIR

Revenir aux données et aux conditions d’essai avant de transporter un résultat vers sa propre machine.